Robust Migration of Java Services to Virtual Threads by AI Agents with Deterministic Code Transformations

Робастная миграция Java-сервисов на виртуальные потоки ИИ-агентами с детерминированными трансформациями кода

Mikhail Trifonov

Virtual threads are one property away, and agents already do the migrating

2023

Virtual threads become final in JDK 21

2024

Netflix services hang on pinned threads

2024

With JEP 491 synchronized stops pinning, from JDK 24 on

2025

Amazon Q upgrades apps to Java 21, Google migrates code with LLMs

2026

Four benchmarks of Java migration by agents

> `spring.threads.virtual.enabled=true`
>
> One line switches Spring Boot to virtual threads. Thread-pool properties then have no effect

> **JDK 21 to 23**
>
> Blocking inside synchronized still holds the carrier thread

Pressler & Bateman, JEP 444, OpenJDK 2023 · Filanovsky et al., Netflix Tech Blog 2024 · Chilano Mateo & Bateman, JEP 491, OpenJDK 2024 · Beach, AWS DevOps Blog 2025 · Nikolov et al., ICSE-SEIP 2025 · Ziftci et al., FSE Companion 2025 · Liu et al., KDD 2026; May et al., ICSE 2026; Amin et al., arXiv 2026; Pavuluri et al., arXiv 2026 · Spring Boot Reference Documentation

Benchmarks count build and tests. None runs the service under load.

	Build	Tests	Similarity to reference	Under load
MigrationBench 5,102 repositories	●	●	○	○
FreshBrew 228 repositories	●	●	○	○
JMigBench function pairs	○	○	●	○
ScarfBench 204 tasks	●	●	○	○
				0 of 4

28.9%

drop of pass@k when HumanEval got 80 times more tests. Weak tests overrate code

Liu et al. (MigrationBench), KDD 2026 · May et al. (FreshBrew), ICSE 2026 · Amin et al. (JMigBench), arXiv 2026 · Pavuluri et al. (ScarfBench), arXiv 2026 · Liu et al. (EvalPlus), NeurIPS 2023

Seven hazards. Each one still builds and passes the tests

PIN

Blocking inside synchronized holds the carrier thread

LIMIT

A pool that protected a resource becomes unbounded

POOL

Virtual threads put into a fixed-size pool

TLOCAL

A costly ThreadLocal cache rebuilt for every virtual thread

CONF

Virtual threads not enabled, or a property that does not exist

DEP

A library version with a known virtual-thread problem

DAEMON

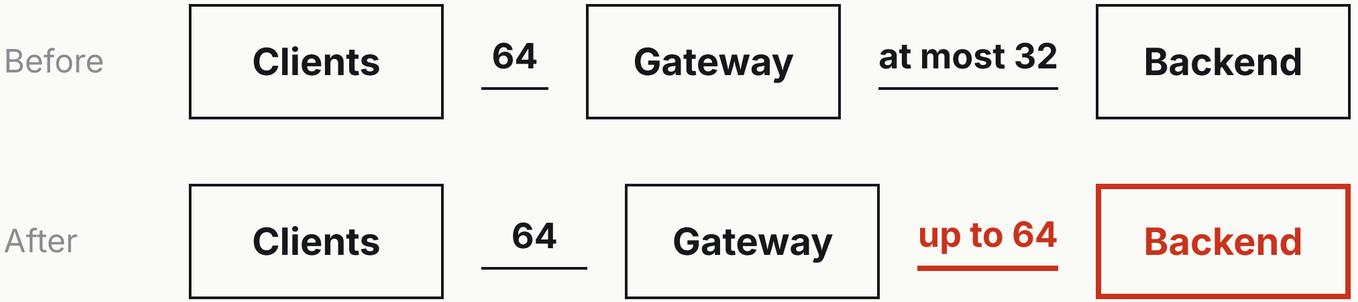
A scheduler-only service exits because virtual threads are daemons

Seen in production

Netflix saw hangs from pinning. HikariCP 7.0.2 saturates all carriers, fixed in 7.1.0. PostgreSQL JDBC 42.7.8 pins

Pressler & Bateman, JEP 444, 2023 · Chilano Mateo & Bateman, JEP 491, 2024 · Spring Boot Reference Documentation · Filanovsky et al., Netflix Tech Blog 2024 · HikariCP issue #2398, 2026 · pgjdbc issue #3835, 2025

The limit lived in a pool setting



application.properties

```
server.tomcat.threads.max=32
+ spring.threads.virtual.enabled=true
```

No Java code changed and the tests stay green. With virtual threads on, thread-pool properties have no effect, so 64 clients reach the backend that was sized for 32.

Spring Boot Reference Documentation: SpringApplication, virtual threads (thread-pool properties have no effect)

Robust migration

“The degree to which a system or component can function correctly in the presence of invalid inputs or stressful environmental conditions”

Safety is a different question. In the dependability taxonomy it means the absence of catastrophic consequences

1

Requests run on virtual threads

thread dump

2

No carrier held in application code

JFR pinning events

3

A resource gets no more concurrent calls than before

counting stub

4

Service alive, at most 1% errors

load run report

5

No library version with a known problem

dependency list

ISO/IEC/IEEE 24765:2010, robustness (via SEBoK glossary) · Avižienis et al., IEEE TDSC 2004 · Definition: own operationalisation (thesis, Ch. 2)

Three groups of work and what I take from each

	Migration benchmarks	Model-driven modernization	Code transformation
What it studies	How well AI agents migrate Java code, from version upgrades to framework changes	Modernizing a system by extracting a model of it and transforming that model. A mapping study found 30 primary studies	Changing code by rules that are written once and give the same result on every run
Main works	MigrationBench, FreshBrew, JMigBench, ScarfBench	OMG ADM, KDM (ISO/IEC 19506), MoDisco	Coccinelle, Refaster, Spoon, Comby, OpenRewrite
Where it stops	**0 of 4** run the service under load. They judge by build, tests or similarity to a reference	**Java 5** is the last Java version MoDisco follows, so it cannot read a service on Java 21	**1 property** is all the OpenRewrite virtual-thread recipe sets. A rule cannot judge the intent of code
What I take	**Real code for my dataset, and the test verdict that I compare with robustness**	**The pipeline of vtmigrate, which extracts a compact model, links its elements to actions and generates code**	**Deterministic edits in vtmigrate. OpenRewrite and Comby also serve as baselines**

Liu et al., KDD 2026 · May et al., ICSE 2026 · Amin et al., arXiv 2026 · Pavuluri et al., arXiv 2026 · ISO/IEC 19506:2012 · Durelli et al., IEEE IRI 2014 · Brunelière et al., IST 2014 · Eclipse MoDisco 1.6.0 review, 2026 · Padioleau et al., EuroSys 2008 · Wasserman, WRT 2013 · Pawlak et al., SPE 2015 · van Tonder & Le Goues, PLDI 2019 · OpenRewrite recipe catalog

Who solved similar problems

Work	What they did	Data	Result	Limit for my question
MigrationBench KDD 2026	Agents migrate Java 8 to 17 or 21	5,102 repos, 300 selected	A Sonnet 4.5 agent reached 71.67% minimal and 53.33% maximal migration	Build and tests only, no load
FreshBrew ICSE 2026	Agents migrate projects to JDK 17	228 repos with high test coverage	Best model migrated 52.3%	Build and tests; one run per task
ScarfBench arXiv 2026	Migration between Spring, Jakarta EE, Quarkus	204 tasks	Best agent passed 15.3% of tests on single-layer tasks	Behaviour, not load; no concurrency
Allain et al. ASE 2026	GPT-5.4, gpt-oss-120B, Llama 3.1 8B write Comby, GritQL, Ast-Grep rules	6 datasets, 4 evolution tasks	Largest model best; small ones struggle with complex migrations	Model writes the rule; I limit it to a choice
MM-assist ICSME 2025	Model proposes Move Method, IDE applies it	2,016 model suggestions (formative study)	Up to 80% of raw suggestions hallucinated	Refactoring, not migration; no load
Google FSE 2025, ICSE-SEIP 2025	Model edits, tools locate and validate	Internal code, 32 to 64-bit IDs	Migration time reduced significantly (experience report)	Model writes code; no controlled comparison

Liu et al., KDD 2026 · May et al., ICSE 2026 · Pavuluri et al., arXiv 2026 · Allain et al., ASE 2026 · Bellur et al., ICSME 2025 · Ziftci et al., FSE Companion 2025 · Nikolov et al., ICSE-SEIP 2025 · Baltes et al., EMSE 2026 (one run per task in FreshBrew, ScarfBench)

What is missing

	Virtual threads	Engine makes every edit	Model only decides	Checked under load
Agent benchmarks MigrationBench, FreshBrew, ScarfBench	○	○	○	○
Model writes rules MELT, Allain et al., BigBag	○	●	○	○
Model picks the action EM-Assist, MM-assist	○	●	●	○
Model edits, tools validate Ziftci et al. (Google)	○	○	○	○
Rules without a model OpenRewrite recipe	◐	●	n/a	○
This thesis vtmigrate + vtcheck	●	●	●	●

Still open is which split of work gives robust migrations and how small the model can be

Liu et al., KDD 2026 · May et al., ICSE 2026 · Pavuluri et al., arXiv 2026 · Ramos et al., ASE 2023 · Allain et al., ASE 2026 · Reyes et al., arXiv 2026 · Pomian et al., FSE 2024 · Bellur et al., ICSME 2025 · Ziftci et al., FSE 2025 · OpenRewrite recipe catalog · ● yes ◐ partly ○ no

Research questions

		Hypothesis
RQ1	**When an AI agent edits the code itself, how often is the migration robust? Which hazards stay hidden behind green tests?** Measured by the share of runs that pass the tests but fail vtcheck, per hazard kind	Many runs pass the tests and are still not robust
RQ2	**Which split of work between model and tools gives robust migrations, and from what model size does each split work?** Measured by the robust share per approach and model on the Qwen 0.8B to 9B ladder	If the model only picks actions, a smaller model is enough
RQ3	**What does each approach cost, and does it give the same result when repeated?** Measured by tokens, model calls or agent steps, seconds and distinct outcomes over repeats	The tool costs about ten times less than an agent and varies less
RQ4	**Do the approaches work on real code that I did not write?** Measured by constructs found and transformed and decisions matching labels on MigrationBench modules	Most constructs are handled; errors come from intent, not from transformations

Questions and hypotheses: thesis Ch. 1 (H1 to H4)

Existing tools and their role

OpenRewrite

The EnableVirtualThreads recipe only sets the property

Baseline without a model

Comby 1.8.1

Rewrites code by declarative templates

Engine for rules a model writes

JavaParser 3.28.2

Parses Java into a syntax tree and edits it

Engine of vtmigrate

Claude Code

Agent harness that reads, edits and runs commands

Agent runs

JFR + Ollama

Pinning events in the JVM. Local open-weight models

Signals and model runtime

MoDisco 1.6.0 + ATL

Classic ADM pipeline

Rejected because it reads no Java after 5

OpenRewrite recipe catalog (EnableVirtualThreads) · Comby 1.8.1, comby.dev · JavaParser 3.28.2, GitHub · OpenJDK JFR event metadata · Ollama model library · Eclipse MoDisco 1.6.0 release review, 2026 · Jouault et al. (ATL), SCP 2008

Ten approaches compared with one check

model does less — model does more

No model	Picks from a compact model	Picks from source code	Writes Comby rules	Rewrites files	Agent
OpenRewrite, generic Comby rules, vtmigrate rules	**vtmigrate**	as in EM-Assist	as in MELT	one call	plain, with a guide, with vtcheck as at Google

vtmigrate, where the model only decides

1. **Extract.** A compact model of pools, monitors, ThreadLocals and limits, each with its allowed actions
2. **Decide.** One model call picks one action per element
3. **Apply.** JavaParser transformations make every edit. A wrong plan changes nothing

vtcheck, the definition of done

Build	Tests	Static rules	Load run

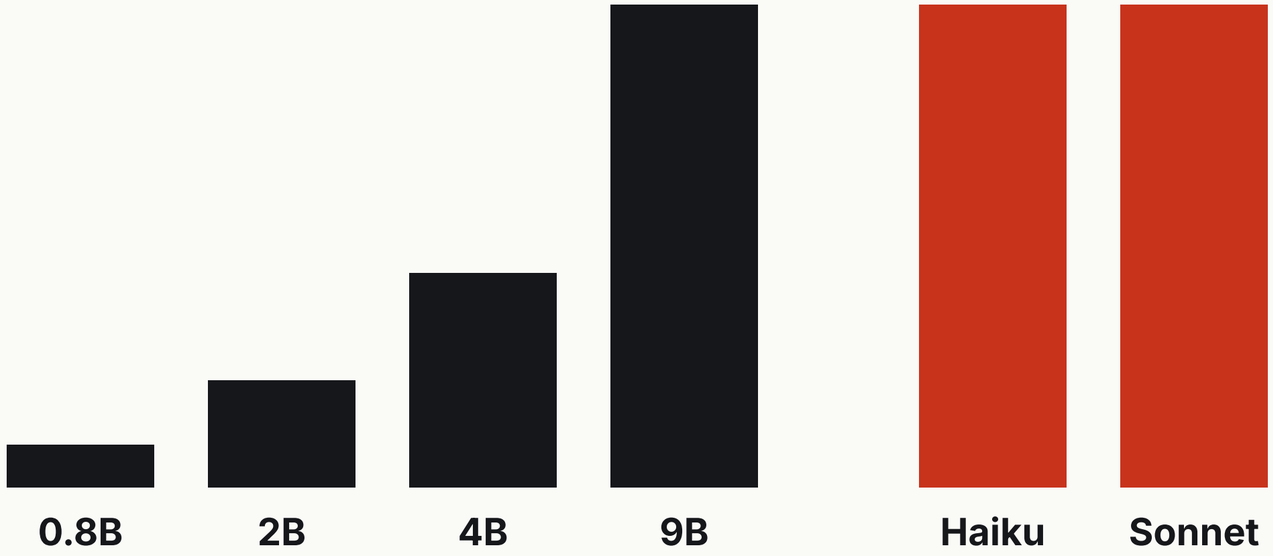
The load run has 64 clients and checks the 5 conditions of robustness

Validated before any run on variants with a known answer, which gives its precision and recall

Weaker on small synthetic services, with real code checked only statically, on one machine, with one annotator

OpenRewrite recipe catalog · Ramos et al. (MELT), ASE 2023 · Pomian et al. (EM-Assist), FSE Companion 2024 · Ziftci et al., FSE Companion 2025 · ISO/IEC 19506:2012 · Del Fabro & Valduriez, SoSyM · JavaParser 3.28.2 · OpenJDK JFR event metadata · Jiang & Hassan, IEEE TSE 2015 · Barr et al., IEEE TSE 2015 · Huang et al., HotOS 2017 · Approaches and repeats: thesis Ch. 3

Models form a ladder of sizes



Qwen3.5 via Ollama

One family, open weights, free on a laptop, so 30 repeats are affordable. Guidelines ask for an open-weight model

Claude Haiku 4.5, Sonnet 5.5

The upper bound, in every approach with a model. The tool also runs on Kimi K3

Agents use 9 models

Haiku, Sonnet, gpt-oss 20B and 120B, Kimi K3, Nemotron 3 Nano, Super, Ultra, Gemma 4 31B, from 5 vendors, in one harness

Ollama model library: qwen3.5 · Baltes et al., EMSE 2026 (include an open-weight model) · Belcak et al., arXiv 2025 (small models for agents) · Model cards, thesis Ch. 3

One migration, on my benchmark and on real code

10 **Spring Boot services**
With 7 hazard kinds labelled and a reference migration. They run under load

≥ 30 **modules from MigrationBench**
Picked by a script with a fixed seed from 5,102 repositories. Size in lines of code and model elements. Checked statically

1 Show the effect
Agent edits the code vs vtmigrate, on the benchmark

2 Explain it
The other splits of work on the same tasks

3 Generalise
Real code from MigrationBench, with a static check

Runs multiply, so the effect comes first. Repeats follow cost, with 30 for the tool, 10 for other one-call approaches, 3 for agents and 1 without a model

Out of scope are general agents, security, reactive stacks, JDK 24 and later, and choosing limit sizes

Liu et al. (MigrationBench), KDD 2026 · MigrationBench dataset, Hugging Face (Apache-2.0) · Chilano Mateo & Bateman, JEP 491, 2024 (JDK 24+ out of scope) · Scope and repeats: thesis Ch. 1 and Ch. 3

What is counted and what it means

Metric	How it is computed	Meaning
Robust share	`robust runs / all runs, Wilson 95% interval`	How often the method gives a robust migration
False success	`runs passing tests but not robust / runs passing tests`	How much the test verdict overstates (RQ1)
Usable answers	`valid plans, rules or files / runs`	Does the model keep the output format
Size boundary	`smallest model not significantly below the rule plan (Fisher, p < 0.05)`	How small the model can be
Cost	`input + output tokens; model calls or agent turns; seconds per run`	What one migration costs
Consistency	`distinct plans and distinct outcomes over the repeats of a task`	1 means fully repeatable
Real code	`found / labelled constructs; applied / found; actions = label / labelled`	Coverage and agreement on MigrationBench code

Wilson, JASA 1927 · Fisher's exact test · Georges et al., OOPSLA 2007 · Kalibera & Jones, ISMM 2013 · Baltes et al., EMSE 2026 (LLM study guidelines)

What will exist at the end

Artifact	Form	Input → output	What it is for
vtmigrate	Java command-line tool, one JAR, on JavaParser	`extract PROJECT` → compact model (JSON) `apply PROJECT plan.json` → changed code and config, report	Migrate a service. The plan comes from any LLM or from built-in rules
vtcheck	Python command-line script	`vtcheck.py PROJECT --task task.json --json report.json` → findings and load-run metrics	Benchmark verdict, tool for an agent, step in CI
Benchmark	10 Spring Boot services and their generator	Each service has code before migration, a reference migration and task.json with labelled hazards	Runnable tasks for any migration method
Dataset	≥30 modules from MigrationBench and selection scripts	Selection with a fixed seed, and labels.json with the correct action and rationale per element	Real-code check of any method
Harness and data	Run scripts for every approach, analysis script	Every run keeps its diff, log, verdict, tokens and time, which become tables and figures	Replicate or extend the study

Repository: bench/vtmigrate, bench/vtcheck.py, bench/make_tasks.py and tasks/, bench/dataset/, bench/run_*.py and analyze.py · Liu et al. (MigrationBench), KDD 2026

What is new compared with prior work

	This thesis	Prior work
Division of work	The model picks one action per element from a closed list over a compact model. Deterministic transformations make every edit	Model writes rules or code in Allain et al., MELT and at Google
Verdict	Robustness judged by runtime resources under load, namely pinning, concurrent calls and liveness	All 4 migration benchmarks stop at build, tests or similarity
Comparison	All splits of work on the same tasks with one check; Haiku and Sonnet in every approach with a model	Each work studies one split
Model size	Boundary inside one family, Qwen3.5 0.8B to 9B	GPT-5.4, gpt-oss-120B, Llama 3.1 8B (Allain et al.)

Allain et al., ASE 2026 · Ramos et al., ASE 2023 · Ziftci et al., FSE Companion 2025 · Liu et al., KDD 2026; May et al., ICSE 2026; Amin et al., arXiv 2026; Pavuluri et al., arXiv 2026

References

1 Allain A. et al. Code Transformation Rule Synthesis using LLMs: Potential and Limits. ASE 2026

2 Amin N. et al. JMigBench: A Benchmark for Evaluating LLMs on Source Code Migration (Java 8 to Java 11). arXiv:2602.09930, 2026

3 Avižienis A. et al. Basic Concepts and Taxonomy of Dependable and Secure Computing. IEEE TDSC, 2004

4 Baltes S. et al. Guidelines for Empirical Studies in Software Engineering involving LLMs. arXiv:2508.15503, accepted in EMSE, 2026

5 Barr E. T. et al. The Oracle Problem in Software Testing: A Survey. IEEE TSE, 2015

6 Beach B. Announcing support for upgrades to Java 21 in Amazon Q Developer. AWS DevOps Blog, 2025

7 Bellur A. et al. Together We are Better: LLM, IDE and Semantic Embedding to Assist Move Method Refactoring. ICSME 2025

8 Belcak P. et al. Small Language Models are the Future of Agentic AI. arXiv:2506.02153, 2025

9 Brunelière H. et al. MoDisco: A Model Driven Reverse Engineering Framework. IST, 2014

10 Chilano Mateo P., Bateman A. JEP 491: Synchronize Virtual Threads without Pinning. OpenJDK, 2024

11 Durelli R. S. et al. A Mapping Study on Architecture-Driven Modernization. IEEE IRI, 2014

12 Eclipse Foundation. Eclipse MoDisco 2026-03 (1.6.0) Release Review, 2026

13 Filanovsky V. et al. Java 21 Virtual Threads: Dude, Where's My Lock? Netflix Technology Blog, 2024

14 Georges A. et al. Statistically Rigorous Java Performance Evaluation. OOPSLA 2007

15 HikariCP issue #2398: yield-spin saturates all carrier threads under virtual thread load. GitHub, 2026

16 ISO/IEC 19506:2012. ADM Knowledge Discovery Meta-Model (KDM)

17 ISO/IEC/IEEE 24765:2010. Systems and software engineering: Vocabulary

18 Jiang Z. M., Hassan A. E. A Survey on Load Testing of Large-Scale Software Systems. IEEE TSE, 2015

19 Huang P. et al. Gray Failure: The Achilles' Heel of Cloud-Scale Systems. HotOS 2017

20 Jouault F. et al. ATL: A Model Transformation Tool. Science of Computer Programming, 2008

References (continued)

21 Kalibera T., Jones R. Rigorous Benchmarking in Reasonable Time. ISMM 2013

22 Liu J. et al. Is Your Code Generated by ChatGPT Really Correct? (EvalPlus). NeurIPS 2023

23 Liu L. et al. MigrationBench: Repository-Level Code Migration Benchmark from Java 8. KDD 2026

24 May V. et al. FreshBrew: A Benchmark for Evaluating AI Agents on Java Code Migration. ICSE 2026

25 Nikolov S. et al. How is Google Using AI for Internal Code Migrations? ICSE-SEIP 2025

26 Padioleau Y. et al. Documenting and Automating Collateral Evolutions in Linux Device Drivers. EuroSys 2008

27 Pavuluri A. et al. ScarfBench: Cross-Framework Application Migration in Enterprise Java. arXiv:2605.06754, 2026

28 Pawlak R. et al. Spoon: A Library for Implementing Analyses and Transformations of Java Source Code. SPE, 2015

29 pgjdbc issue #3835: Virtual Thread pinned in PostgreSQL JDBC Driver 42.7.8. GitHub, 2025

30 Pomian D. et al. EM-Assist: Safe Automated ExtractMethod Refactoring with LLMs. FSE Companion 2024

31 Pressler R., Bateman A. JEP 444: Virtual Threads. OpenJDK, 2023

32 Ramos D. et al. MELT: Mining Effective Lightweight Transformations from Pull Requests. ASE 2023

33 Reyes F. et al. Agentic Generation of AST Transformation Rules for Fixing Breaking Updates. arXiv:2606.24446, 2026

34 Spring Boot Team. Spring Boot Reference Documentation: SpringApplication; Task Execution and Scheduling

35 van Tonder R., Le Goues C. Lightweight Multi-Language Syntax Transformation with Parser Parser Combinators. PLDI 2019

36 Wasserman L. Scalable, Example-Based Refactorings with Refaster. WRT 2013

37 Wilson E. B. Probable Inference, the Law of Succession, and Statistical Inference. JASA, 1927

38 Ziftci C. et al. Migrating Code At Scale With LLMs At Google. FSE Companion 2025

39 Tools: OpenRewrite recipe catalog; Comby 1.8.1; JavaParser 3.28.2; OpenJDK JFR metadata; Ollama qwen3.5; MigrationBench dataset (Hugging Face)